

Polygon Fill: Jordan Curve Animation

概覽

Lab10 在 Lab09 Jordan curve fill 的理論基礎上，加入逐步動畫系統，讓使用者能在互動式網格上觀察 odd-even crossing test 對每個像素的分類過程。

整個畫面分為兩個區域：

- 網格區：主要視覺區，顯示多邊形邊界、填色動畫、與當前步驟游標。
- 狀態列：畫面底部，顯示頂點清單、顏色圖例、播放狀態、與當前像素的 crossing 細節。

互動操作

滑鼠

右鍵點擊畫面任意處可開啟選單，選擇網格維度：

選項	維度
10	10 × 10 (預設)
15	15 × 15
20	20 × 20
Custom...	手動輸入整數，按 Enter 確認，Esc 取消

左鍵點擊網格格點可新增頂點；若點擊同一格點，則移除該頂點。頂點依點擊順序連成多邊形，每次變更後自動重算邊的 Bresenham 光柵化結果與所有動畫步驟。

鍵盤

按鍵	動作
P	正向播放；若正在正向播放則暫停
V	逆向播放；若正在逆向播放則暫停
N / Space	單步前進一格並暫停；已到末端則不動
R	從頭重播 (重設到第 0 步並立即開始正向播放)
+ / =	加速 (每次縮短 30 ms，最短 30 ms)
- / _	減速 (每次延長 30 ms，最長 1000 ms)
Esc	清除所有頂點，重置多邊形

P 與 V 的邏輯為 **toggle-same-direction**：

- 若目前靜止，按 P 開始正向，按 V 開始逆向。
- 若正在正向播放，按 P 暫停；按 V 停止正向並從當前位置開始逆向。
- 若正在逆向播放，按 V 暫停；按 P 停止逆向並從當前位置開始正向。

視覺元素

網格區

元素	顏色	說明
格線	灰色	座標網格，每格代表一個整數格點
邊像素	黑色	Bresenham 光柵化後的邊界像素 (實心正方形)
頂點	彩虹色	依點擊順序循環上色：紅、橙、黃、綠、藍、紫

內部格點	白色	動畫已揭露且 crossing number 為奇數的格點
外部格點	橘紅色	動畫已揭露且 crossing number 為偶數的格點
當前步驟	青色外框	動畫目前正在評估的格點，以矩形邊框標示

狀態列 (底部四行)

第一行：頂點清單

Vertices: N v1=(x,y) ... [N = 維度]

若頂點數 ≤ 4 ，列出每個頂點座標；超過 4 個時只顯示數量。

第二行：顏色圖例

Vertices: rainbow Edge: black Inside(A): white Outside(B): red

第三行：播放狀態

Playback: {status} step X/Y delay=Zms controls: ...

status 的可能值如下：

status	含義
ready	尚未建立任何動畫步驟 (頂點不足或多邊形退化)
playing	正在正向自動播放
reversing	正在逆向自動播放
stepping	手動暫停於中途 (step > 0)
finished	正向播放已到最後一步
reversed	逆向播放已回到第 0 步

X/Y 顯示目前揭露步數 X 與總步數 Y；delay=Zms 顯示目前每幀延遲毫秒數。

第四行：當前像素詳情

Current pixel: (x,y) cross number = C classification = {result}

result 的可能值：outside (偶數 crossing)、inside (奇數 crossing)、boundary (落在邊上)、inside [edge pixel] (odd-even 判為內部但同時是邊界光柵像素)。

若尚未開始播放，則顯示 waiting。

動畫掃描順序

動畫步驟依 由上至下、由左至右 的順序建立，掃描範圍為多邊形 bounding box 向外各延伸 1 格：

$$[x_{\min} - 1, x_{\max} + 1] \times [y_{\min} - 1, y_{\max} + 1].$$

並非 bounding box 內所有格點都成為動畫步驟——系統只保留滿足以下任一條件的格點：

1. odd-even test 判為內部 (cross mod 2 = 1)。
2. 落在光柵化後的邊界上 (visual_boundary = true)。
3. 八鄰域中有任一邊界格點 (顯示邊界外一圈緩衝帶，方便觀察邊界附近的分類結果)。

外部且遠離邊界的格點不納入動畫，以節省步驟數並聚焦在有意義的區域。

使用流程

1. 在網格上依序點擊至少 3 個不共線頂點，建立一個多邊形。
2. 按 P 或 V 開始播放，觀察系統逐格執行 odd-even crossing test 並著色。
3. 按 N 或 Space 逐步手動前進，對照第四行的 crossing number 理解當前格點的分類依據。
4. 按 V 逆向倒回，確認動畫可雙向播放。
5. 用 + / - 調整速度；按 R 從頭重播。
6. 按 Esc 清空後，嘗試凹多邊形或複雜形狀，觀察 odd-even rule 的填色效果。

逐步動畫系統之設計

資料結構

每個掃描格點的完整資訊封裝在 **AnimationStep** 中：

欄位	型別	說明
cell	Cell	格點整數座標 (x, y)
query	OddEvenQueryResult	crossings 次數、on_boundary 旗標、inside 結果
visual_boundary	bool	是否同時是 Bresenham 光柵化邊界像素

所有格點按掃描順序儲存為 `std::vector<AnimationStep>`，播放時以 `revealed_step_count` 作為游標，畫面只繪製索引 `[0, revealed_step_count)` 的格點。

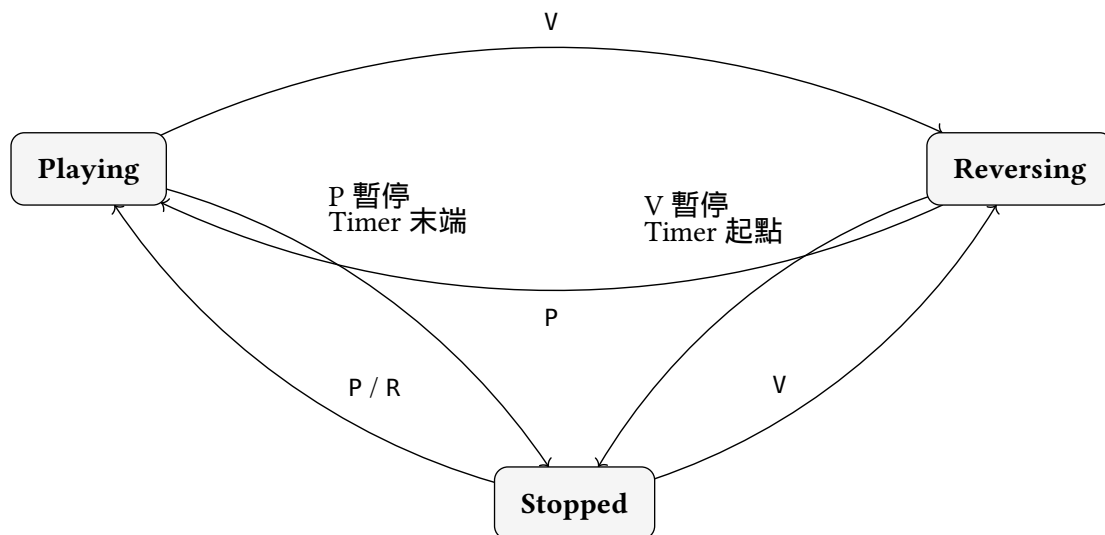
播放控制的所有執行期狀態封裝在 **AnimationPlaybackState** 中：

欄位	型別	說明
mode	PlaybackMode	目前播放模式 (Stopped / Playing / Reversing)
revealed_step_count	size_t	動畫游標，已揭露步數
playback_delay_ms	int	每幀間隔毫秒數，可用 +/- 鍵調整
playback_generation	int	世代計數，用於作廢過期 Timer (見後節)

播放狀態機

原始設計以 `{is_playing, is_reversing}` 兩個 bool 表示播放方向，但這會產生四種組合，其中 `is_playing=false, is_reversing=true` 是語意上不可達的無效狀態。重構後改用 **PlaybackMode** enum，三個值恰好對應三個有效狀態，消除無效組合：

$$\text{PlaybackMode} \in \{\text{Stopped}, \text{Playing}, \text{Reversing}\}$$



N/Space 不論當前處於哪個狀態，均無條件轉移至 Stopped 並將游標前進一步（固定為正向）。
完整轉移規則：

觸發	來源	目標	附加動作
P	Stopped	Playing	若游標已在末端，先重設為 0
P	Playing	Stopped	暫停，游標保留原位
P	Reversing	Playing	停止逆向，從游標當前位置正向繼續
V	Stopped	Reversing	若游標為 0，先跳至末端
V	Reversing	Stopped	暫停，游標保留原位
V	Playing	Reversing	停止正向，從游標當前位置逆向繼續
N / Space	任意	Stopped	若未到末端則游標 += 1（固定正向）
R	任意	Playing	游標重設為 0，從頭正向播放
Timer 觸發	Playing	Stopped	游標抵達末端，mode = Stopped
Timer 觸發	Reversing	Stopped	游標抵達 0，mode = Stopped

Generation Counter：消除過期 Timer

GLUT 的 `glutTimerFunc` 是 **fire-and-forget**——Timer 一旦送出即無法撤銷。若使用者在 Timer 到期前切換狀態（例如按 P 暫停再按 V 開始逆向），佇列中殘存的舊 Timer 仍會在延遲後觸發，造成幽靈步進。

解決方式：引入單調遞增的 **generation counter** - `playback_generation`，以「版本號比對」取代「撤銷」：

- 啟動或停止播放時，`stop_animation` 執行 `++playback_generation`，令 generation counter 遞增。
- 排程新 Timer 時，將當時的 generation counter 作為回呼參數送入：

```
glutTimerFunc(delay_ms, &App::timer_cb, playback_generation);
```

- `timer_cb` 收到回呼後，先核對 generation:

```
if (mode == Stopped || generation != playback_generation) return;
```

若 generation 不符，代表這是過期 Timer，直接丟棄，不執行任何步進。

如此，無論佇列中殘留多少舊 Timer，只有攜帶最新 generation counter 的那一個會實際執行。timer_cb 在非終止幀的最後排程下一個 Timer，形成單鏈自我排程：

$$\text{timer}_n \rightarrow \text{step} \rightarrow \text{timer}_{n+1} \rightarrow \dots \rightarrow \text{timer}_{\text{end}} \rightarrow \text{Stopped}$$

動畫步驟建立

每當多邊形發生任何頂點異動，build_animation_steps 被同步呼叫，一次性計算所有步驟並快取於 animation_steps 向量：

1. 以 compute_bounding_box 計算頂點 bounding box，向外各擴展 1 格得掃描範圍：

$$[x_{\min} - 1, x_{\max} + 1] \times [y_{\min} - 1, y_{\max} + 1].$$

2. 按由上至下、由左至右順序遍歷每個格點，呼叫 query_point_odd_even 取得 crossing 結果。
3. 過濾：僅保留下列三類格點，略去純外部格點以縮短序列長度：
 1. query.inside == true (odd-even 判為內部)
 2. visual_boundary == true (是光柵化邊界像素)
 3. has_boundary_neighbor 回傳 true (八鄰域含邊界像素，形成緩衝帶)
4. 通過過濾的格點以 AnimationStep{cell, query, visual_boundary} 形式推入向量。

播放時 revealed_step_count 每幀遞增（正向）或遞減（逆向），畫面在 draw_revealed_fill 中逐一繪製揭露格點，即呈現逐步著色的動畫效果。